

Laboratory Component 1(a): Hello World

Question:

Write a script that logs "Hello, World!" to the console.

```
<!DOCTYPE html>

<html>

  <body>

    <h1>My First Web Page</h1>

    <script>

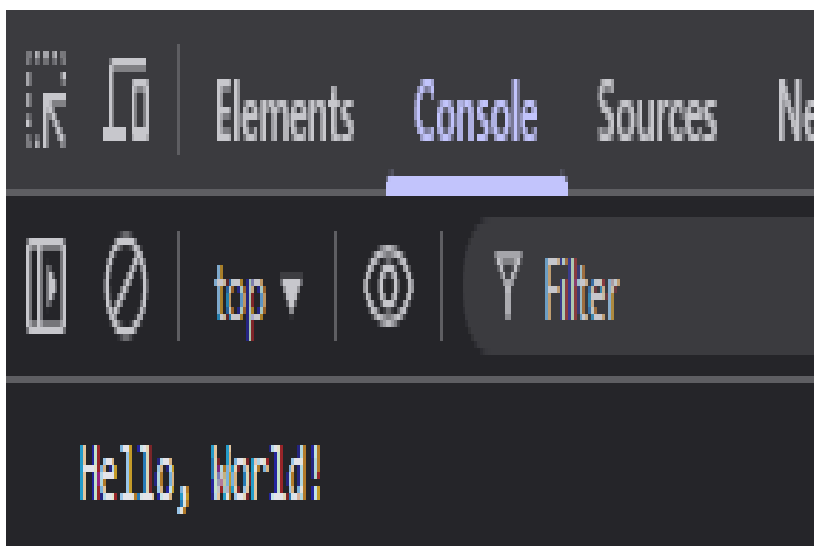
      console.log("Hello, World! (Frontend)");

    </script>

  </body>

</html>
```

Output:



Laboratory Component 1(b): Sum of Two Numbers

Create a script that calculates the sum of two numbers and displays the result in an alert box.

```
<!DOCTYPE html>

<html>

<body>

  <script>

    let num1 = 5;

    let num2 = 7;

    let sum = num1 + num2;

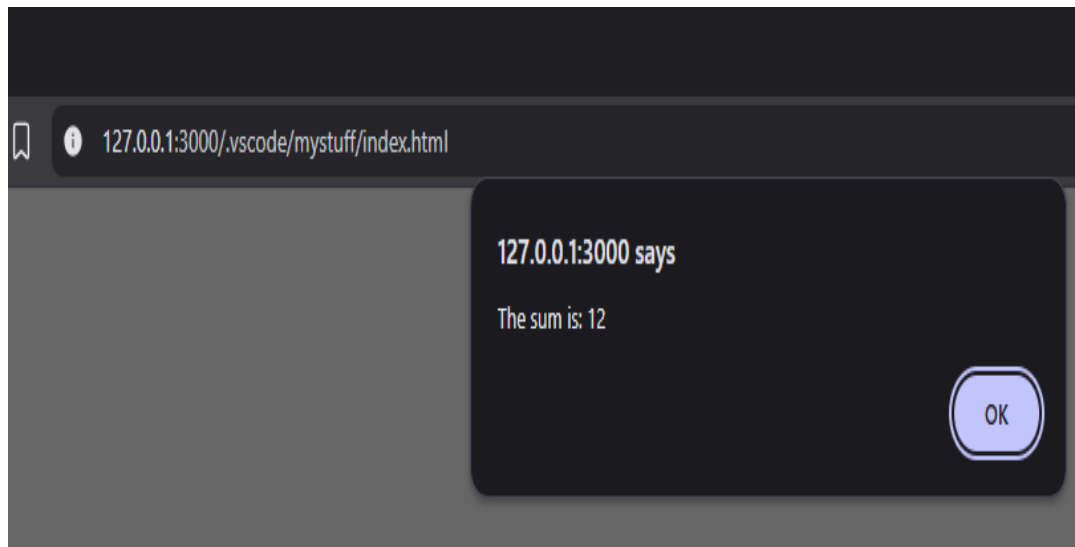
    alert("The sum is: " + sum);

  </script>

</body>

</html>
```

Output:



Laboratory Component 2: Array of Cities

Question:

Create an array of 5 cities and perform the following operations:

- **Log the total number of cities.**
- **Add a new city at the end.**
- **Remove the first city.**
- **Find and log the index of a specific city.**

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>City Array Operations</title>

</head>

<body>

    <h1>City Array Operations</h1>

    <p>Current Cities: <span id="cityList"></span></p>

    <!-- Buttons for operations -->

    <button onclick="logTotalCities()">Log Total Number of Cities</button>

    <button onclick="addCity()">Add a New City</button>

    <button onclick="removeFirstCity()">Remove First City</button>

    <button onclick="findCity()">Find a Specific City</button>

    <script>

        // Initial array of cities

        let cities = ["New York", "London", "Tokyo", "Paris", "Sydney"];
```

```
let cityList = document.getElementById("cityList");  
cityList.textContent = cities.join(", ");
```

```
// Log the total number of cities
```

```
function logTotalCities() {  
    alert("Total number of cities: " + cities.length);  
}
```

```
// Add a new city
```

```
function addCity() {  
    let newCity = prompt("Enter a city to add:");  
    if (newCity) {  
        cities.push(newCity);  
        cityList.textContent = cities.join(", ");  
        alert(newCity + " added!");  
    }  
}
```

```
// Remove the first city
```

```
function removeFirstCity() {  
    if (cities.length > 0) {  
        let removedCity = cities.shift();  
        cityList.textContent = cities.join(", ");  
        alert(removedCity + " removed!");  
    } else {  
        alert("No cities left to remove.");  
    }  
}
```

```
}
```

```
//4 Find a specific city
```

```
function findCity() {
```

```
    let cityToFind = prompt("Enter the city to find:");
```

```
    let index = cities.indexOf(cityToFind);
```

```
    if (index !== -1) {
```

```
        alert(cityToFind + " found at index " + index);
```

```
    } else {
```

```
        alert(cityToFind + " not found in the array.");
```

```
    }
```

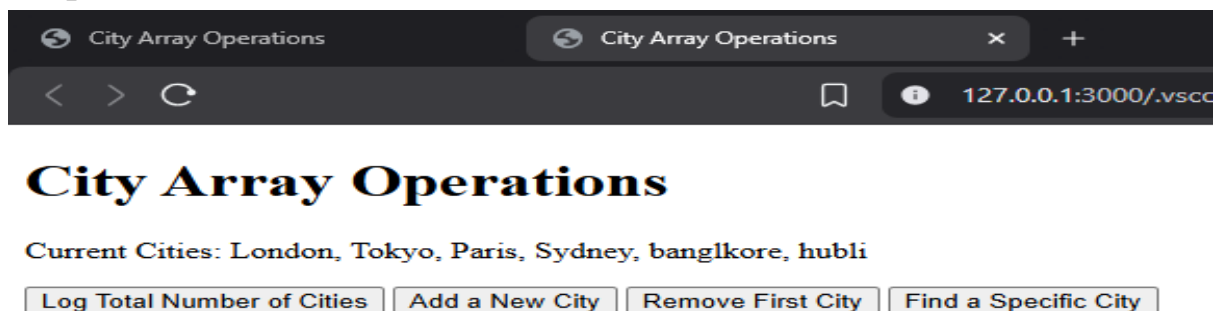
```
}
```

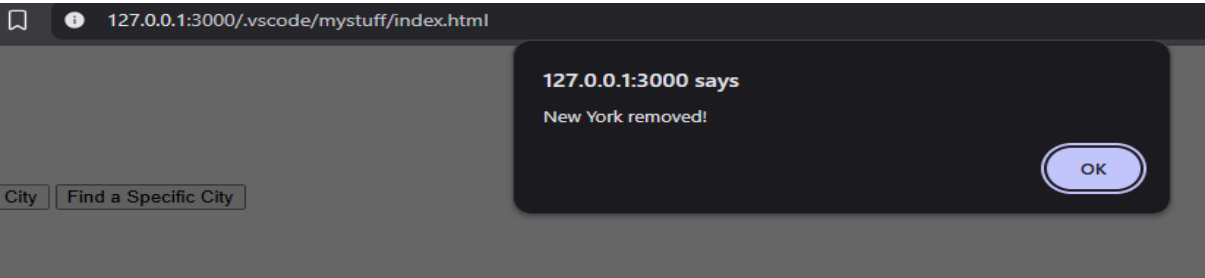
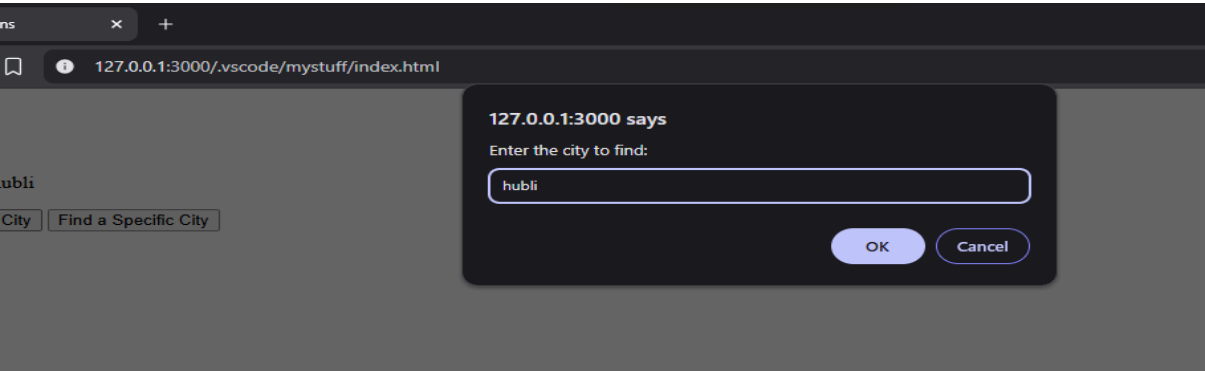
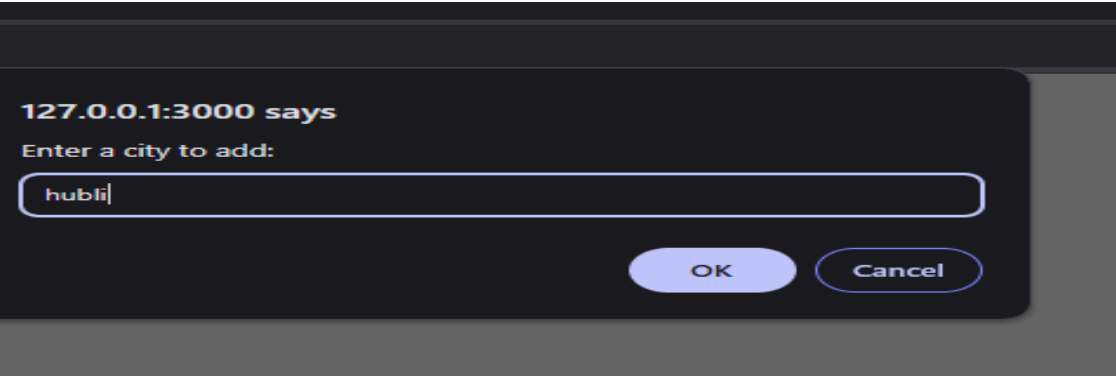
```
</script>
```

```
</body>
```

```
</html>
```

Output:





Laboratory Component 3: String Operations and Palindrome Check

Question:

Read a string from the user, find its length. Extract the word "JavaScript" using `substring()` or `slice()`. Replace one word with another word and log the new string. Write a function `isPalindrome(str)` that checks if a given string is a palindrome (reads the same backward).

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>String Operations & Palindrome Check</title>
```

```
</head>
```

```
<body>
```

```
  <h1>String Operations and Palindrome Check</h1>
```

```
  <label for="userString">Enter a string:</label>
```

```
  <input type="text" id="userString" placeholder="Type something here">
```

```
  <br><br>
```

<button onclick="performStringOperations()">Perform String Operations</button>

<button onclick="checkPalindrome()">Check Palindrome</button>

<h3>Results:</h3>

<p id="lengthResult"></p>

<p id="substringResult"></p>

<p id="replaceResult"></p>

<p id="palindromeResult"></p>

<script>

// Function to perform string operations

function performStringOperations() {

let str = document.getElementById("userString").value;

if (!str) {

alert("Please enter a string.");

return;

}

//① Find the length of the string

let length = str.length;

document.getElementById("lengthResult").textContent = "Length of the string: " +
length;

//② Extract the word "JavaScript"

let startIndex = str.indexOf("JavaScript");

let extracted = "";

```

if (startIndex !== -1) {
    extracted = str.slice(startIndex, startIndex + "JavaScript".length);
} else {
    extracted = "JavaScript not found in the string.";
}

document.getElementById("substringResult").textContent = "Extracted word: " +
extracted;

```

// 3 Replace a word with another

```

let wordToReplace = prompt("Enter the word to replace:");
let newWord = prompt("Enter the new word:");
if (wordToReplace && newWord) {
    let newStr = str.replace(wordToReplace, newWord);
    document.getElementById("replaceResult").textContent = "New string: " + newStr;
} else {
    document.getElementById("replaceResult").textContent = "Replacement skipped.";
}
}

```

// Function to check palindrome

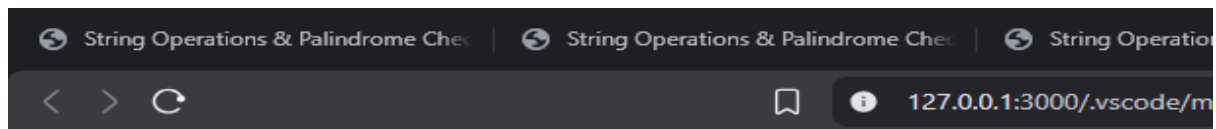
```

function isPalindrome(str) {
    // Remove spaces and convert to lowercase for accurate comparison
    let cleaned = str.replace(/\s+/g, "").toLowerCase();
    let reversed = cleaned.split("").reverse().join("");
    return cleaned === reversed;
}

```

```
function checkPalindrome() {  
    let str = document.getElementById("userString").value;  
    if (!str) {  
        alert("Please enter a string.");  
        return;  
    }  
    let result = isPalindrome(str) ? "Yes, it is a palindrome!" : "No, it is not a  
palindrome.";  
    document.getElementById("palindromeResult").textContent = result;  
}  
</script>  
</body>  
</html>
```

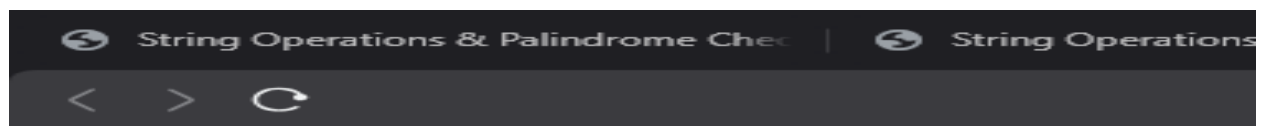
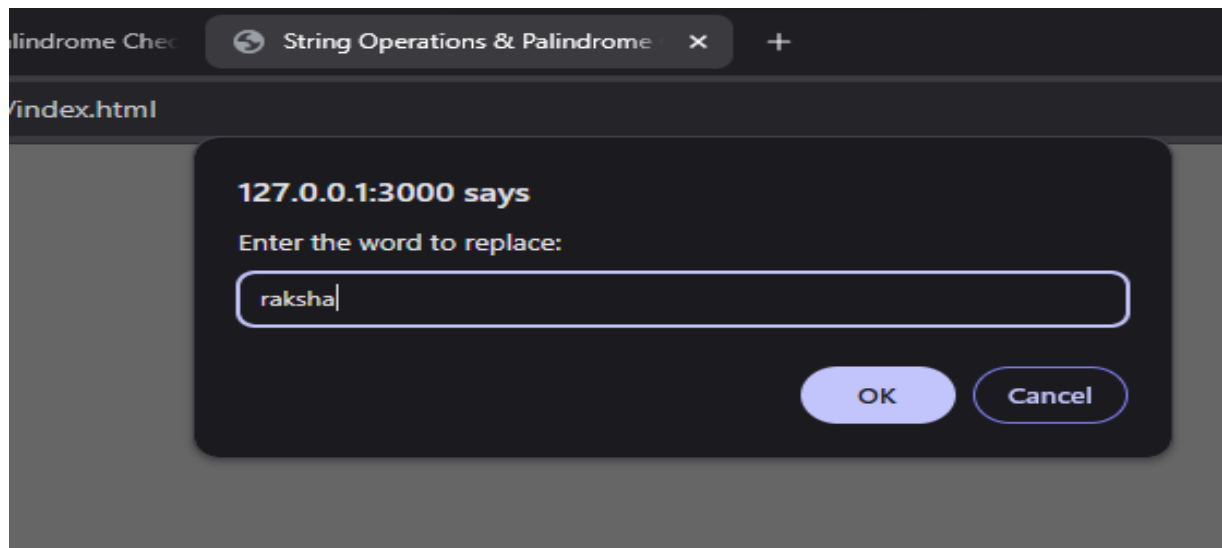
Output:



String Operations and Palindrome Check

Enter a string:

Results:



String Operations and Palindrome

Enter a string:

Results:

Length of the string: 4

Extracted word: JavaScript not found in the string.

Replacement skipped.

Yes, it is a palindrome!

Laboratory Component 4: Student Object Operations

Question:

Create an object student with properties: name (string), grade (number), subjects (array), displayInfo() (method to log the student's details). Write a script to dynamically add a passed property to the student object, with a value of true or false based on their grade. Create a loop to log all keys and values of the student object.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Student Object Operations</title>

</head>

<body>

  <h1>Student Object Operations</h1>

  <label for="studentName">Name:</label>

  <input type="text" id="studentName" placeholder="Enter student name"><br><br>

  <label for="studentGrade">Grade:</label>

  <input type="number" id="studentGrade" placeholder="Enter student grade"><br><br>

  <label for="studentSubjects">Subjects (comma separated):</label>

  <input type="text" id="studentSubjects" placeholder="e.g.
Math,English,Science"><br><br>

  <button onclick="createStudent()">Create Student</button>
```

```
<h3>Student Info:</h3>
```

```
<pre id="studentInfo"></pre>
```

```
<script>
```

```
    let student;
```

```
    function createStudent() {
```

```
        // Get input values
```

```
        const name = document.getElementById("studentName").value;
```

```
        const grade = parseFloat(document.getElementById("studentGrade").value);
```

```
        const subjects = document.getElementById("studentSubjects").value.split(",").map(s  
=> s.trim());
```

```
        if (!name || isNaN(grade) || subjects.length === 0) {
```

```
            alert("Please fill in all fields correctly.");
```

```
            return;
```

```
        }
```

```
        //  Create student object
```

```
        student = {
```

```
            name: name,
```

```
            grade: grade,
```

```
            subjects: subjects,
```

```
            displayInfo: function() {
```

```
                return `Name: ${this.name}\nGrade: ${this.grade}\nSubjects:  
${this.subjects.join(", ")} `;
```

```
            }
```

```
        };
```

//**2** Dynamically add 'passed' property (assuming passing grade ≥ 50)

```
student.passed = grade  $\geq$  50;
```

//**3** Loop through all keys and values

```
let output = "";
```

```
for (let key in student) {
```

```
  if (student.hasOwnProperty(key)) {
```

```
    output += `${key}: ${student[key]}\n`;
```

```
  }
```

```
}
```

```
// Display result
```

```
document.getElementById("studentInfo").textContent = output;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Output:

Student Object Operations

Name:

Grade:

Subjects (comma separated):

Student Info:

```
name: shri
grade: 4
subjects: maths.science
displayInfo: function() {
    return `Name: ${this.name}\nGrade: ${this.grade}\nSubjects: ${this.subjects.join(", ")}`;
}
passed: false
```

Laboratory Component 5: DOM Events

Question:

Create a button in your HTML with the text "Click Me". Add an event listener to log "Button clicked!" to the console when the button is clicked. Select an image and add a mouseover event listener to change its border color. Add an event listener to the document that logs the key pressed by the user.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>DOM Events Lab</title>

  <style>

    #myImage {

      width: 200px;

      height: auto;

      border: 3px solid black;

      transition: border-color 0.3s;

    }

  </style>

</head>

<body>

  <h1>DOM Events Example</h1>

  <!--1 Button -->
```

```
<button id="myButton">Click Me</button>
```

```
<br><br>
```

```
<!--Image -->
```

```

```

```
<p>Hover over the image to change its border color.</p>
```

```
<script>
```

```
// Button click event
```

```
const button = document.getElementById("myButton");
```

```
button.addEventListener("click", () => {
```

```
  console.log("Button clicked!");
```

```
});
```

```
// Image mouseover event
```

```
const image = document.getElementById("myImage");
```

```
image.addEventListener("mouseover", () => {
```

```
  image.style.borderColor = "red"; // Change border color
```

```
});
```

```
image.addEventListener("mouseout", () => {
```

```
  image.style.borderColor = "black"; // Revert border color
```

```
});
```

```
// Document keypress event
```

```
document.addEventListener("keydown", (event) => {
```

```
  console.log("Key pressed:", event.key);
```

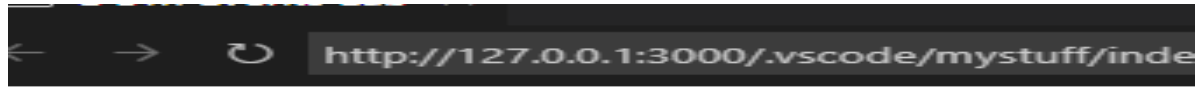
```
});
```

```
</script>
```

```
</body>
```

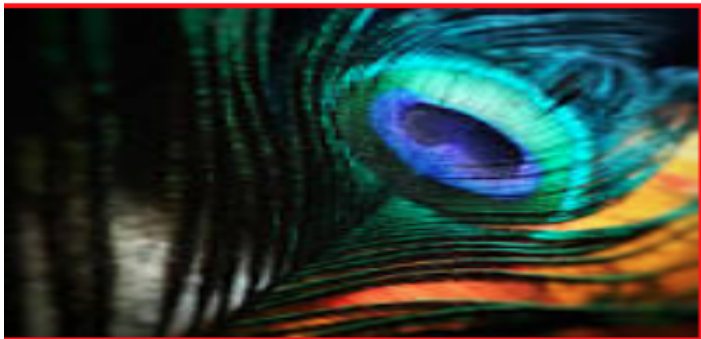
```
</html>
```

Output:



DOM Events Example

Click Me



Hover over the image to change its border color.

Laboratory Component 6: React Issue Tracker

Question:

Build a React application to track issues. Display a list of issues (use static data). Each issue should have a title, description, and status (e.g., Open/Closed). Render the list using a functional component.

```
import React from "react";

function IssueRow(props) {
  const issue = props.issue;
  return (
    <tr>
      <td>{issue.id}</td>
      <td>{issue.status}</td>
      <td>{issue.owner}</td>
      <td>{issue.created.toString()}</td>
      <td>{issue.effort}</td>
      <td>{issue.due ? issue.due.toString() : ""}</td>
      <td>{issue.title}</td>
    </tr>
  );
}

function IssueTable(props) {
  const issueRows = props.issues.map((issue) => (
    <IssueRow key={issue.id} issue={issue} />
  ));
  return (
```

```

<table border="1" style={{ marginTop: "10px" }}>
  <thead>
    <tr>
      <th>ID</th>
      <th>Status</th>
      <th>Owner</th>
      <th>Created</th>
      <th>Effort</th>
      <th>Due Date</th>
      <th>Title</th>
    </tr>
  </thead>
  <tbody>{issueRows}</tbody>
</table>
);
}

```

```

class App extends React.Component {
  constructor() {
    super();
    this.state = {
      issues: [
        {
          id: 1,
          status: "Open",
          owner: "Ashok",
          created: new Date("2025-10-01"),

```

```

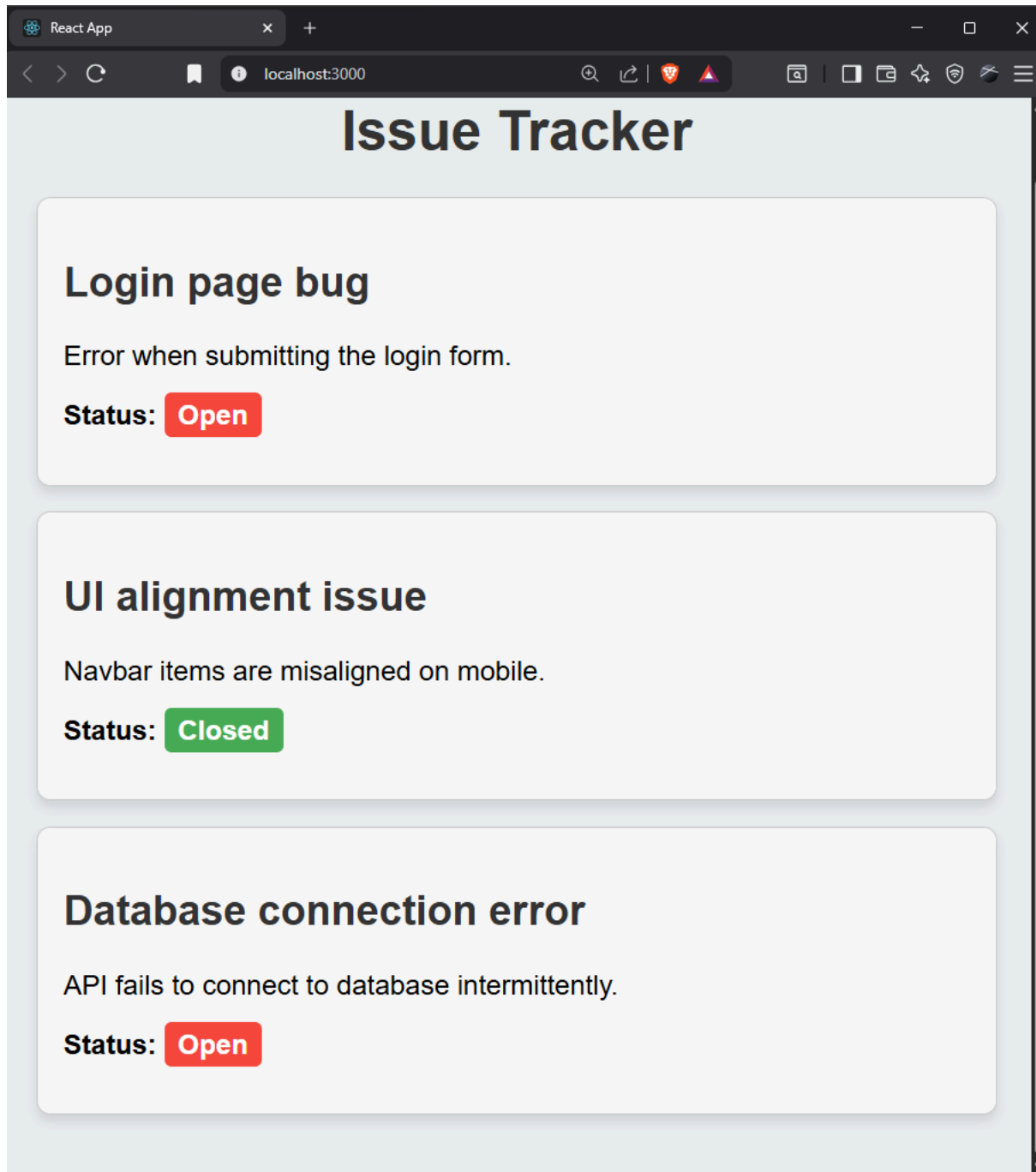
        effort: 5,
        due: new Date("2025-10-20"),
        title: "Fix login issue",
    },
    {
        id: 2,
        status: "Assigned",
        owner: "Rahul",
        created: new Date("2025-10-03"),
        effort: 3,
        due: new Date("2025-10-18"),
        title: "Update dashboard UI",
    },
],
};
}

render() {
    return (
        <div>
            <h2>Issue Tracker – Stateless Components Example</h2>
            <IssueTable issues={this.state.issues} />
        </div>
    );
}
}

```

```
export default App;
```

Output:



Laboratory Component 7: React Counter Component

Question:

Create a component Counter with a state variable count initialized to 0. Create buttons to increment and decrement the count. Simulate fetching initial data for the Counter component using useEffect (functional component) or componentDidMount (class component). Extend the Counter component to double the count value when a button is clicked. Reset the count to 0 using another button.

APP.js

```
import React from "react";  
import Counter from "../Counter";
```

```
function App() {  
  return (  
    <div className="App">  
      <Counter />  
    </div>  
  );  
}
```

```
export default App;
```

counter.jsx

```
import React, { useState, useEffect } from "react";
```

```
export default function Counter() {
```

```
// state variable 'count' initialized to 0
const [count, setCount] = useState(0);

// simulate fetching initial data on mount
useEffect(() => {
  console.log("Simulating fetch of initial count...");

  const timerId = setTimeout(() => {
    setCount(5); // pretend API returned 5
  }, 1000);

  // cleanup in case component unmounts before timeout
  return () => clearTimeout(timerId);
}, []); // empty dependencies means run once on mount

// handlers using functional updates (safer with async updates)
const increment = () => setCount((c) => c + 1);
const decrement = () => setCount((c) => c - 1);
const double = () => setCount((c) => c * 2);
const reset = () => setCount(0);

return (
  <div style={{ textAlign: "center", marginTop: 40 }}>
    <h2>Counter Value: {count}</h2>

    <div style={{ display: "inline-flex", gap: 8 }}>
      <button type="button" onClick={increment}>Increment</button>
```

```
<button type="button" onClick={decrement}>Decrement</button>

<button type="button" onClick={double}>Double</button>

<button type="button" onClick={reset}>Reset</button>

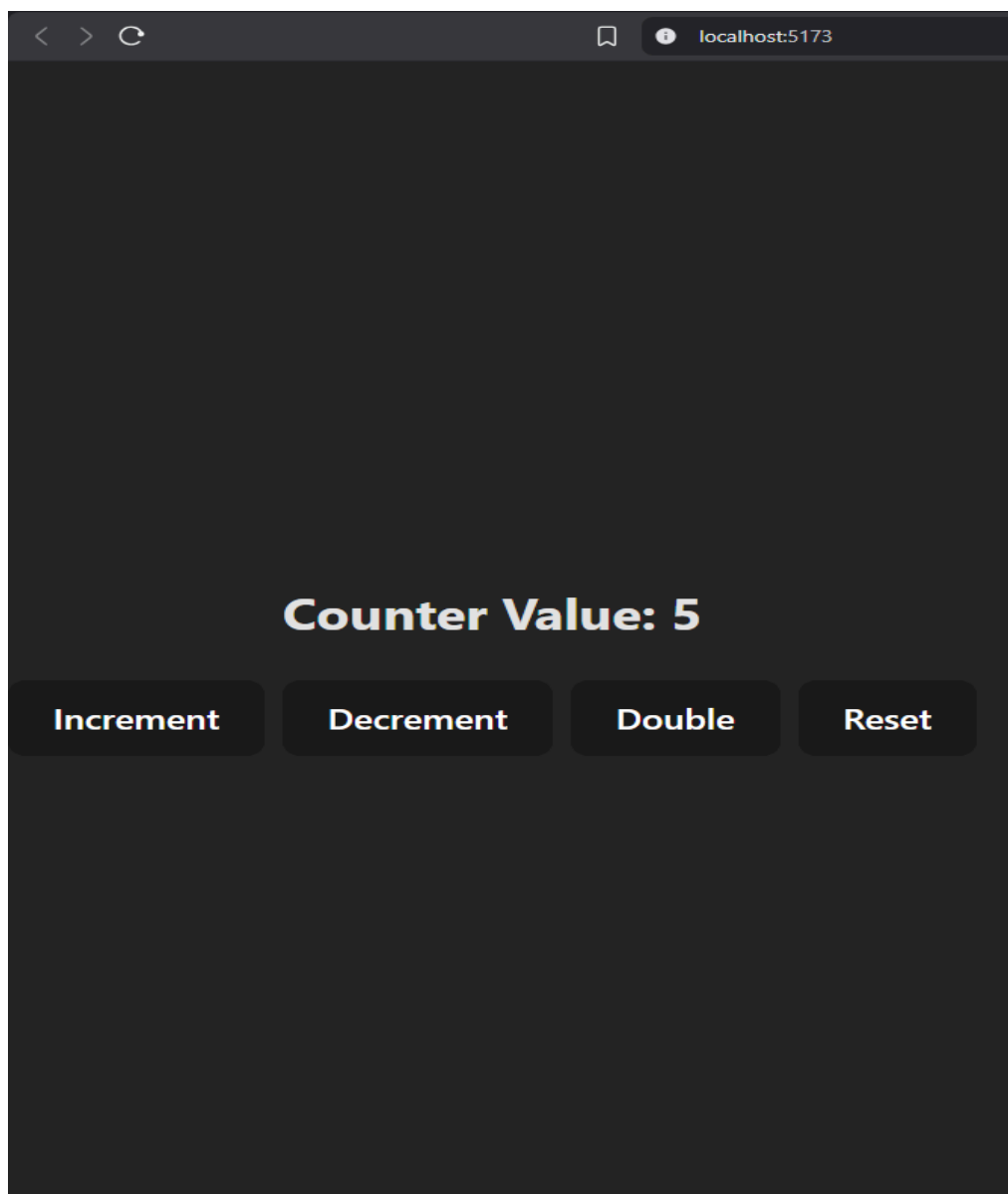
</div>

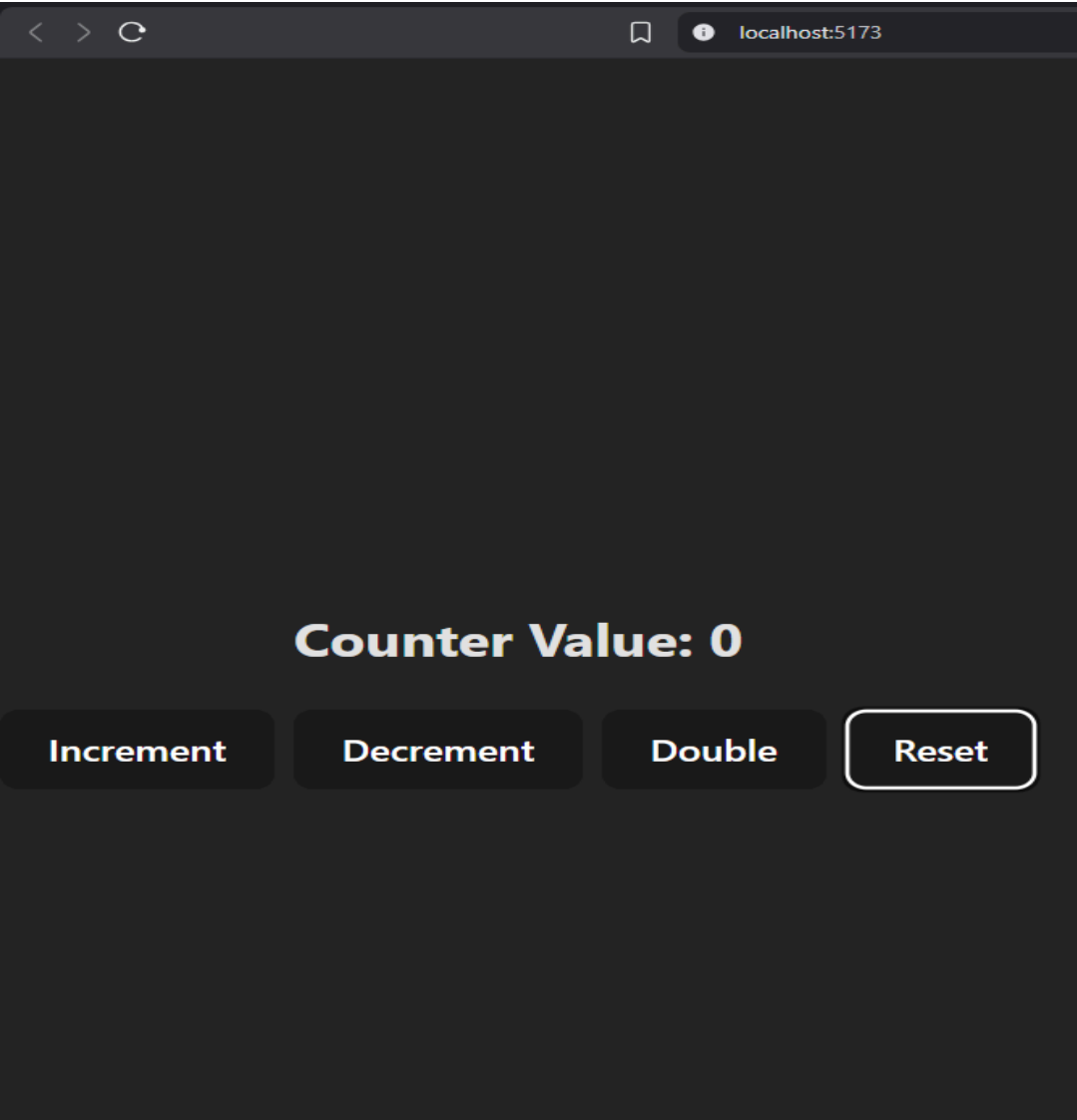
</div>

);

}
```

Output:





Laboratory Component 8: Basic Express Server

Question:

Install Express (npm install express). Set up a basic server that responds with "Hello, Express!" at the root endpoint (GET /).

```
const express = require('express');
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// In-memory product store
```

```
let products = [];
```

```
let nextId = 1;
```

```
// Middleware to parse JSON
```

```
app.use(express.json());
```

```
// Middleware to log requests
```

```
app.use((req, res, next) => {
```

```
  console.log(`${req.method} ${req.url}`);
```

```
  next();
```

```
});
```

```
// Root endpoint
```

```
app.get('/', (req, res) => {
```

```
  res.send('Hello, Express!');
```

```
});
```

```
// POST endpoint to add products
```

```
app.post('/products', (req, res) => {
```

```
  const product = {
```

```
    id: nextId++,
```

```
    name: req.body.name,
```

```
  };
```

```
  products.push(product);
```

```
  res.status(201).json(product);
```

```
});
```

```
// Start the server
```

```
app.listen(PORT, () => {  
  console.log(`Server is running on http://localhost:${PORT}`);  
});
```

Output:



Laboratory Component 9: Express REST API

Question:

Create a REST API. Implement endpoints for a Product resource:

- **GET /products:** Returns a list of products.
- **POST /products:** Adds a new product.
- **GET /products/:id:** Returns details of a specific product.
- **PUT /products/:id:** Updates an existing product.
- **DELETE /products/:id:** Deletes a product.

Add middleware to log requests to the console. Use `express.json()` to parse incoming JSON payloads.

```
const express = require('express');

const app = express();

app.use(express.json());

app.use((req, res, next) => {

  console.log(`${req.method} ${req.url}`);

  next();

});
```

```
let products = [

  { id: 1, name: 'Laptop', price: 60000 },

  { id: 2, name: 'Mobile', price: 20000 },

];
```

```
app.get('/products', (req, res) => {

  res.json(products);

});
```

```
app.post('/products', (req, res) => {

  const newProduct = {
```

```

    id: products.length + 1,
    name: req.body.name,
    price: req.body.price
  };
  products.push(newProduct);
  res.status(201).json(newProduct);
});

app.put('/products/:id', (req, res) => {
  const product = products.find(p => p.id === parseInt(req.params.id));
  if (!product) return res.status(404).json({ message: 'Product not found' });
  product.name = req.body.name;
  product.price = req.body.price;
  res.json(product);
});

app.delete('/products/:id', (req, res) => {
  const productIndex = products.findIndex(p => p.id === parseInt(req.params.id));
  if (productIndex === -1) return res.status(404).json({ message: 'Product not found' });
  const deletedProduct = products.splice(productIndex, 1);
  res.json(deletedProduct[0]);
});

const PORT = 3000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

```

Output:

DEL http://localhost:5000/pre

POST http://localhost:3000

GET http://localhost:3000/pre

+

...

http://localhost:3000/products/

Save

GET http://localhost:3000/products/ Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (7) Test Results

Status: 200 OK Time: 8 ms Size: 275 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 4,
3   "name": "Tablet",
4   "price": 15000
5 }
```

http://localhost:3000/products/1

Save

PUT http://localhost:3000/products/1 Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON

Beautify

```
1 {
2   "name": "Tablet",
3   "price": 15000
4 }
5
```

Body Cookies Headers (7) Test Results

Status: 404 Not Found Time: 4 ms Size: 273 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Product not found"
3 }
```

DEL http://localhost:5000/pro...

POST http://localhost:3000

DEL http://localhost:3000/pro...

+

...

http://localhost:3000/products/2

Save

DELETE

http://localhost:3000/products/2

Send

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

JSON

1

2

3

4

5

"name": "Tablet",

"price": 15000

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 5 ms

Size: 273 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

"id": 2,

"name": "Tablet",

"price": 15000

http://localhost:3000/products/

Save

POST

http://localhost:3000/products/

Send

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

JSON

1

2

3

4

5

"name": "Tablet",

"price": 15000

Body

Cookies

Headers (7)

Test Results

Status: 201 Created

Time: 4 ms

Size: 278 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

"id": 2,

"name": "Tablet",

"price": 15000

DEL http://localhost:5000/pro... POST http://localhost:3000 DEL http://localhost:3000/pro... + ...

http://localhost:3000/products/2

DELETE http://localhost:3000/products/2 Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Tablet",
3   "price": 15000
4 }
5
```

Body Cookies Headers (7) Test Results Status: 404 Not Found Time: 3 ms Size: 273 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Product not found"
3 }
```

DEL http://localhost:5000/pro... POST http://localhost:3000 PUT http://localhost:3000/pro... + ...

http://localhost:3000/products/5

PUT http://localhost:3000/products/5 Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Tablet",
3   "price": 15000
4 }
5
```

Body Cookies Headers (7) Test Results Status: 404 Not Found Time: 3 ms Size: 273 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Product not found"
3 }
```

Activate Windows

Laboratory Component 10: MongoDB Connection

Question:

Install the MongoDB driver for Node.js. Create a Node.js script to connect to the shop database. Implement insert, find, update, and delete operations using the Node.js MongoDB driver.

```
// shop.js

const { MongoClient } = require("mongodb");

// Connection URL and Database Name
const url = "mongodb://127.0.0.1:27017";
const dbName = "shop"; // Always use lowercase

async function main() {
  const client = new MongoClient(url);

  try {
    // Connect to MongoDB
    await client.connect();
    console.log("✅ Connected successfully to MongoDB");

    // Check for any old conflicting database name ("Shop")
    const admin = client.db().admin();
    const dbList = await admin.listDatabases();

    const hasOldDb = dbList.databases.some(db => db.name === "Shop");
    if (hasOldDb) {
```

```
console.log("⚠ Found old database 'Shop' — dropping it to avoid conflict...");  
await client.db("Shop").dropDatabase();  
console.log("🧹 Dropped old database 'Shop'.");  
}
```

```
// Use the lowercase 'shop' database
```

```
const db = client.db(dbName);  
const collection = db.collection("products");
```

```
// Clear existing documents (optional)
```

```
await collection.deleteMany({});
```

```
// INSERT
```

```
const insertResult = await collection.insertMany([  
  { name: "Laptop", price: 60000, quantity: 5 },  
  { name: "Mouse", price: 800, quantity: 50 },  
  { name: "Keyboard", price: 1500, quantity: 30 }  
]);  
console.log(`🟢 Inserted ${insertResult.insertedCount} documents.`);
```

```
// FIND
```

```
const findResult = await collection.find({}).toArray();  
console.log("📦 Found documents:", findResult);
```

```
// UPDATE
```

```
const updateResult = await collection.updateOne(  
  { name: "Laptop" },
```

```

    { $set: { price: 58000 } }
  );
  console.log(`✎ Updated ${updateResult.modifiedCount} document(s).`);

  // DELETE

  const deleteResult = await collection.deleteOne({ name: "Mouse" });
  console.log(`🗑 Deleted ${deleteResult.deletedCount} document(s).`);

  // FINAL STATE

  const finalDocs = await collection.find({}).toArray();
  console.log(`📊 Final collection data:", finalDocs);

} catch (err) {
  console.error("❌ Error:", err.message);
} finally {
  await client.close();
  console.log("🔒 Connection closed");
}
}

main();

output:

```

```
PS C:\Users\Lenovo\Desktop\4MT23CS197\lab10> npm init -y
>> npm install mongodb
Wrote to C:\Users\Lenovo\Desktop\4MT23CS197\lab10\package.json:

{
  "name": "lab10",
  "version": "1.0.0",
  "description": "",
  "main": "shop.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

```

PS C:\Users\Lenovo\Desktop\4MT23CS197\lab10> node shop.js
✓ Connected successfully to MongoDB
⚠ Found old database 'Shop' – dropping it to avoid conflict...
✂ Dropped old database 'Shop'.
● Inserted 3 documents.
📦 Found documents: [
  {
    _id: new ObjectId('6915779b2a322f20a47562c3'),
    name: 'Laptop',
    price: 60000,
    quantity: 5
  },
  {
    _id: new ObjectId('6915779b2a322f20a47562c4'),
    name: 'Mouse',
    price: 800,
    quantity: 50
  },
  {
    _id: new ObjectId('6915779b2a322f20a47562c5'),
    name: 'Keyboard',
    price: 1500,
    quantity: 30
  }
]
✏ Updated 1 document(s).
🗑 Deleted 1 document(s).
📋 Final collection data: [
  {
    _id: new ObjectId('6915779b2a322f20a47562c3'),
    name: 'Laptop',
    price: 58000,
    quantity: 5
  },
  {
    _id: new ObjectId('6915779b2a322f20a47562c5'),
    name: 'Keyboard',
    price: 1500,
    quantity: 30
  }
]
🔒 Connection closed

```

Laboratory Component 11: Mongoose Product Schema

Question:

Define a product schema using Mongoose. Insert data into the products collection using Mongoose. Create an Express API with a /products endpoint to fetch all products.

```
// server.js

// Import dependencies

const express = require("express");

const mongoose = require("mongoose");

// Initialize Express app

const app = express();

app.use(express.json());

// Connect to MongoDB (modern syntax — no deprecated options)

mongoose.connect("mongodb://127.0.0.1:27017/shop")

  .then(() => console.log("✅ Connected to MongoDB"))

  .catch(err => console.error("❌ MongoDB connection error:", err));

// Define Mongoose Schema and Model

const productSchema = new mongoose.Schema({

  name: String,

  category: String,

  price: Number,

  inStock: Boolean

});
```

```

const Product = mongoose.model("Product", productSchema);

// Insert sample data (only once)
async function insertSampleData() {
  try {
    const count = await Product.countDocuments();
    if (count === 0) {
      await Product.insertMany([
        { name: "Laptop", category: "Electronics", price: 50000, inStock: true },
        { name: "Phone", category: "Electronics", price: 20000, inStock: false },
        { name: "Shoes", category: "Fashion", price: 1500, inStock: true }
      ]);
      console.log("🟢 Sample products inserted!");
    }
  } catch (err) {
    console.error("❌ Error inserting sample data:", err.message);
  }
}

// Run sample data insertion after DB connects
mongoose.connection.once("open", () => {
  insertSampleData();
});

// ===== CRUD ROUTES =====

// READ - Get all products

```

```
app.get("/products", async (req, res) => {  
  try {  
    const products = await Product.find();  
    res.json(products);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

// CREATE - Add a new product

```
app.post("/products", async (req, res) => {  
  try {  
    const product = new Product(req.body);  
    await product.save();  
    res.status(201).json(product);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
});
```

// UPDATE - Modify a product by ID

```
app.put("/products/:id", async (req, res) => {  
  try {  
    const product = await Product.findByIdAndUpdate(req.params.id, req.body, { new: true });  
    if (!product) return res.status(404).json({ message: "Product not found" });  
    res.json(product);  
  } catch (err) {
```

```
    res.status(400).json({ error: err.message });
  }
});

// DELETE - Remove a product by ID
app.delete("/products/:id", async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) return res.status(404).json({ message: "Product not found" });
    res.json({ message: "Product deleted successfully" });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ===== START SERVER ===== //
const PORT = 3000;
app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`));
```

```

PS C:\Users\Lenovo\Desktop\4MT23CS197\lab11> mkdir product-api
>> cd product-api
>> npm init -y
● >> npm install express mongoose

Directory: C:\Users\Lenovo\Desktop\4MT23CS197\lab11

Mode                LastWriteTime         Length Name
----                -
d-----          13-11-2025    11:55             product-api
Wrote to C:\Users\Lenovo\Desktop\4MT23CS197\lab11\product-api\package.json:

{
  "name": "product-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

added 85 packages, and audited 86 packages in 3s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

○ PS C:\Users\Lenovo\Desktop\4MT23CS197\lab11\product-api>

```

```

Connected to MongoDB
PS C:\Users\Lenovo\Desktop\4MT23CS197\lab11> node server.js
>>
🚀 Server running on port 3000
✅ Connected to MongoDB

```

GET

▼

http://localhost:3000/products

Params

Authorization

Headers (10)

Body ●

Pre-request Script

Tests

Settings

Query Params

	Key
	Key

Body

Cookies

Headers (7)

Test Results

Pretty

Raw

Preview

Visualize

JSON ▼

≡

```
1  [
2    {
3      "_id": "6915779b2a322f20a47562c3",
4      "name": "Laptop",
5      "price": 58000,
6      "quantity": 5
7    },
8    {
9      "_id": "6915779b2a322f20a47562c5",
10     "name": "Keyboard",
11     "price": 1500,
12     "quantity": 30
13   }
14 ]
```

Laboratory Component 12: React Fetch to Express API

Question:

Use fetch in React to call the POST /products endpoint in Express to insert a new product. Update the Product List: After adding a product, update the list of products displayed in React.

```
// server.js

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Connect MongoDB (modern syntax)
mongoose.connect('mongodb://127.0.0.1:27017/shop')
  .then(() => console.log('✅ MongoDB Connected'))
  .catch(err => console.log('❌ MongoDB connection error:', err));

// Product Schema
const productSchema = new mongoose.Schema({
  name: String,
  price: Number,
  category: String,
```

```
});
```

```
const Product = mongoose.model('Product', productSchema);
```

```
// ROUTE 1: Get all products
```

```
app.get('/products', async (req, res) => {  
  try {  
    const products = await Product.find();  
    res.json(products);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

```
// ROUTE 2: Add a new product
```

```
app.post('/products', async (req, res) => {  
  try {  
    const { name, price, category } = req.body;  
    const newProduct = new Product({ name, price, category });  
    await newProduct.save();  
    res.status(201).json(newProduct);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
});
```

```
// ROUTE 3: Update a product by ID
```

```
app.put('/products/:id', async (req, res) => {  
  try {  
    const updatedProduct = await Product.findByIdAndUpdate(  
      req.params.id,  
      req.body,  
      { new: true }  
    );  
    if (!updatedProduct) return res.status(404).json({ message: 'Product not found' });  
    res.json(updatedProduct);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
});
```

// ROUTE 4: Delete a product by ID

```
app.delete('/products/:id', async (req, res) => {  
  try {  
    const deletedProduct = await Product.findByIdAndDelete(req.params.id);  
    if (!deletedProduct) return res.status(404).json({ message: 'Product not found' });  
    res.json({ message: 'Product deleted successfully' });  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
});
```

// Start server

```
const PORT = 5000;
```

```
app.listen(PORT, () => console.log( Server running on port ${PORT} `));
```

```

PS C:\Users\Lenovo\Desktop\4MT23CS197\backend> npm init -y
>> npm install express mongoose cors
Wrote to C:\Users\Lenovo\Desktop\4MT23CS197\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

added 87 packages, and audited 88 packages in 3s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Lenovo\Desktop\4MT23CS197\backend>

```

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5000/products`
- Method:** `POST`
- Body Type:** `JSON`
- Request Body:**

```

1  {
2    "name": "Gaming Laptop",
3    "price": 75000
4  }

```
- Response Body (Pretty):**

```

1  {
2    "_id": "6915779b2a322f20a47562c3",
3    "name": "Laptop",
4    "price": 58000,
5    "quantity": 5
6  },
7  {
8    "_id": "6915779b2a322f20a47562c5",
9    "name": "Keyboard",
10   "price": 1500,
11   "quantity": 30
12 }

```

